	Politechnika Bydgoska im. J. J. Śniadeckich Wydział Telekomunikacji, Informatyki i Elektrotechniki		
Przedmiot	Zaawansowane programowanie obiektowe		
Prowadzący	dr inż. Damian Szczegielniak		
Temat	Implementacja wzorca Producent – Konsument generującego statystyki słów z wykorzystaniem programowania funkcyjnego i współbieżnego		
Student	Jakub Ziółkowski, grupa 1, Informatyka stosowana, IV semestr, studia stacjonarne		
Nr ćw.	1	Data wykonania	19.02.2026 r.
Ocena		Data oddania spr.	24.03.2026 r.

Spis treści

1. Cel laboratorium.....	2
2. Rozwiązanie	2
2.1. Ogólny opis aplikacji	2
2.2. Pula wątków i zarządzanie zasobami.....	3
2.3. Kolejka blokującą - komunikacja między producentem a konsumentami	4
2.4. Zadanie podstawowe - zliczanie słów: uzupełnione fragmenty TODO dla producenta i konsumenta	4
2.4.1. Pierwszy fragment TODO - przekazanie poison pills konsumentom	4
2.4.2. Drugi fragment TODO - wyszukiwanie plików *.txt i wstawianie do kolejki	5
2.4.3. Trzeci fragment TODO 3 - pobieranie ścieżki i tworzenie statystyki wyrazów	6
2.5. Przetwarzanie tekstu - metoda getLinkedCountedWords i strumienie.....	7
2.6. Zadanie dodatkowe - wyznaczanie podobieństwa kosinusowego	8
2.6.1. Obsługa producenta w zadaniu dodatkowym	8
2.6.2. Konsument - liczenie podobieństwa kosinusowego.....	9
2.6.3. Obliczanie miary kosinusowej	10
2.7. Interfejs graficzny	11
3. Wnioski końcowe (Podsumowanie).....	12

1. Cel laboratorium

Celem tego laboratorium było poznanie bardziej zaawansowanych mechanizmów programowania w Javie, szczególnie związanych z pracą z wieloma wątkami (z programowaniem współbieżnym) w puli (ExecutorService), oraz programowaniem funkcyjnym. Przećwiczyłem również między innymi użycie wyrażień lambda, strumieni (Streams) oraz sposób, w jaki te elementy można połączyć z wzorcem producent–konsument z użyciem kolejki blokującej (LinkedBlockingQueue). Użyłem w programie operatora referencji do metody, zastosowałem metody `isInterrupted()` i `interrupt()` do pracy z przerwaniem, jak również mechanizm `Files.walkFileTree` do przechodzenia przez drzewo katalogów.

Zadanie praktyczne polegało na napisaniu aplikacji, która analizuje pliki tekstowe znajdujące się we wskazanym katalogu (z wykorzystaniem `Files.walkFileTree`) i wyznacza statystykę najczęściej występujących słów, tak aby producent wstawiał ścieżki do plików do kolejki, a konsumenci pobierali te ścieżki i liczyli słowa. Dodatkowo w zadaniu dodatkowym zaimplementowałem obliczanie podobieństwa kosinusowego pomiędzy dokumentami a wybranym dokumentem wzorcowym.

2. Rozwiązanie

2.1. Ogólny opis aplikacji

Aplikację zaimplementowałem w oparciu o wzorec producent–konsument z użyciem kolejki blokującej `LinkedBlockingQueue`. Główna logika sterująca wątkami i kolejką znajduje się w klasie `MainFrame`, która dodatkowo pełni rolę kontrolera GUI (okno z przyciskami `Start/Stop/Cosine/Zamknij` dla zadania dodatkowego). Za przetwarzanie tekstu i liczenie statystyk odpowiada osobna klasa `WordStatisticsService`, dzięki czemu logika przetwarzania plików jest odseparowana od logiki interfejsu i zarządzania wątkami.

W zadaniu podstawowym (`getMultiThreadedStatistics`) działa jeden producent i dwóch konsumentów. Producent przeszukuje drzewo katalogów w poszukiwaniu plików `.txt` i dla każdego znalezionej pliku umieszcza w kolejce obiekt `Optional<Path>` wskazujący na ten plik. Konsumenci pobierają obiekty z kolejki, liczą najczęściej występujące słowa w plikach, a następnie wypisują wyniki w konsoli. Do zakończenia pracy konsumentów użyłem specjalnych obiektów „poison pill” (`Optional.empty()`), które są wrzucane do kolejki przez producenta, kiedy aplikacja ma się zatrzymać.

W zadaniu dodatkowym (`getCosineSimilarityStatistics`) architektura jest podobna, ale konsumenci zamiast statystyki słów liczą podobieństwo kosinusowe względem jednego, wspólnego dokumentu wzorcowego (`pattern.txt`). Ta wersja programu również korzysta z kolejki blokującej i poison pills, ale inny jest sposób przetwarzania pliku przez konsumenta.

2.2. Pula wątków i zarządzanie zasobami

Do zarządzania wątkami użyłem `ExecutorService` z metodą `Executors.newFixedThreadPool`, ustawiając liczbę wątków na sumę liczby producentów i konsumentów. W zadaniu są to 1 producent i 2 konsumentów, czyli trzy wątki robocze:

```
82= public MainFrame() {  
83     liczbaWrazowStatystyki = 10;  
84     fajrant = new AtomicBoolean(false);  
85     liczbaProducentow = 1;  
86     liczbaKonsumentow = 2;  
87     executor = Executors.newFixedThreadPool(liczbaProducentow + liczbaKonsumentow);  
88     producentFuture = new ArrayList<>();  
89     statisticsService = new WordStatisticsService();  
90     initialize();  
91 }
```

Takie podejście pozwala oddać zarządzanie wątkami do `ExecutorService`, dzięki czemu nie ma potrzeby ręcznego tworzenia i niszczenia obiektów typu `Thread`. Do nadzoru pracy producenta używana jest lista `List<Future<?>>` `producentFuture`, do której dodawany jest wynik `executor.submit(producent)` przy uruchamianiu producenta. Dzięki temu można np. sprawdzić, czy jakiś producent nadal działa, zanim wystartuje się nowe zadanie:

```
140     for (Future<?> f : producentFuture) {  
141         if (!f.isDone()) {  
142             JOptionPane.showMessageDialog(frame,  
143                 "Nie można uruchomić nowego zadania! Przynajmniej jeden producent nadal działa!",  
144                 "OSTRZEŻENIE", JOptionPane.WARNING_MESSAGE);  
145             return;  
146         }  
147     }
```

Użycie `AtomicBoolean` `fajrant` pozwala sygnalizować wątkom, że mają kończyć pracę (np. po kliknięciu przycisku „Stop”). Zamknięcie okna aplikacji wywołuje `executor.shutdownNow()`, co przerywa wszystkie wątki i pozwala zakończyć program w kontrolowany sposób.

2.3. Kolejka blokującą - komunikacja między producentem a konsumentami

Do komunikacji między wątkami służy fragment:

```
149 final BlockingQueue<Optional<Path>> kolejka = new LinkedBlockingQueue<>(liczbaKonsumentow);
```

Ustawiłem pojemność kolejki na liczbę konsumentów, dzięki temu jest to niewielka kolejka, która wymusza, żeby producent działał w odpowiednim tempie względem konsumentów. Metoda put() blokuje producenta, gdy kolejka jest pełna, natomiast take() blokuje konsumenta, gdy kolejka jest pusta. Dzięki temu nie trzeba martwić się o synchronizację – kolejka blokująca sama to zapewnia.

Każdy element kolejki to Optional<Path>. Gdy element zawiera ścieżkę (optPath.isPresent()), oznacza to normalne działanie, plik jest przetwarzany. Gdy jest pusty (Optional.empty()), jest traktowany jako „poison pill”, czyli sygnał zakończenia pracy dla konsumenta. Producent odpowiada za wrzucenie tylu poison pills, ilu jest konsumentów, tak żeby każdy dostał swój sygnał.

2.4. Zadanie podstawowe- zliczanie słów: uzupełnione fragmenty TODO dla producenta i konsumenta

2.4.1. Pierwszy fragment TODO - przekazanie poison pills konsumentom

W metodzie getMultiThreadedStatistics() producent działa w pętli, dopóki nie zostanie przerwany albo dopóki nie zostanie ustawiony fajrant. Fragment związany z TODO 1 zaimplementowałem w taki sposób:

```
159 if(fajrant.get()) {
160     // TODO przekazanie poison pills (kolejka.put(Optional.empty());) konsumentom i zakończenia działania
161     info = String.format("PRODUCENT %s - wysyłam poison pills i kończę.", name);
162     System.out.println(info);
163     try {
164         for (int i = 0; i < liczbaKonsumentow; i++) {
165             kolejka.put(Optional.empty());
166         }
167     } catch (InterruptedException e) {
168         System.out.printf("PRODUCENT %s przerwany podczas wysyłania poison pills%n", name);
169         Thread.currentThread().interrupt();
170     }
171     break;
172 }
```

Po ustawieniu fajrant na true (np. po kliknięciu „Stop”) producent zamiast szukać kolejnych plików wrzuca do kolejki Optional.empty() tyle razy, ilu jest konsumentów. Jeśli podczas put() wątek zostanie przerwany, program obsługuje InterruptedException, wypisuje informację w konsoli i ponownie ustawia flagę przerwania, żeby bezpiecznie wyjść z pętli. W ten sposób konsument, który pobierze Optional.empty(), kończy pracę.

2.4.2. Drugi fragment TODO - wyszukiwanie plików *.txt i wstawianie do kolejki

Drugi fragment TODO dotyczył wyszukiwania plików tekstowych i wstawiania ich ścieżek do kolejki. Zaimplementowałem to z użyciem `Files.walkFileTree` i `SimpleFileVisitor<Path>`:

```
173 // TODO Wyszukiwanie plików *.txt i wstawianie do kolejki ścieżki opakowanej w Optional
174 Path dir = Paths.get(DIR_PATH);
175 if (!Files.exists(dir)) {
176     System.out.printf("Katalog %s nie istnieje!\n", DIR_PATH);
177 } else {
178     PathMatcher matcher = FileSystems.getDefault().getPathMatcher("glob:**/*.*txt");
179     Files.walkFileTree(dir, new SimpleFileVisitor<Path>() {
180         @Override
181         public FileVisitResult visitFile(Path file, BasicFileAttributes attrs) throws IOException {
182             if (matcher.matches(file)) {
183                 try {
184                     Optional<Path> optPath = Optional.ofNullable(file);
185                     kolejka.put(optPath);
186                     System.out.printf("PRODUCENT %s dodał do kolejki: %s\n", name, file);
187                 } catch (InterruptedException e) {
188                     System.out.printf("PRODUCENT %s przerwany podczas put()\n", name);
189                     Thread.currentThread().interrupt();
190                     return FileVisitResult.TERMINATE;
191                 }
192             }
193             return FileVisitResult.CONTINUE;
194         }
195     });
196 }
```

Najpierw program sprawdza, czy katalog `DIR_PATH` istnieje; jeśli nie, wypisuje komunikat w konsoli. Następnie tworzy `PathMatcher` z wzorcem `glob:**/*.*txt`, który pozwala wybrać wszystkie pliki zakończone na `.txt` w całym drzewie katalogów.

W `visitFile` sprawdza, czy dany plik pasuje do wzorca, i jeśli tak, opakowuje ścieżkę w `Optional<Path>` i wkłada ją do kolejki za pomocą `kolejka.put(optPath)`. Jeśli `put()` zostanie przerwany, ustawia flagę przerwania i przerywa dalsze przechodzenie po drzewie plików.

W efekcie producent cyklicznie przechodzi po katalogach, wrzucając do kolejki ścieżki do plików tekstowych. Po każdej turze szukania robi przerwę `TimeUnit.SECONDS.sleep(przerwa)`, którą można przerwać, co też jest obsługiwane w kodzie.

2.4.3. Trzeci fragment TODO 3 - pobieranie ścieżki i tworzenie statystyki wyrazów

Ostatni fragment TODO dotyczył strony konsumenta - miał on pobierać ścieżkę z kolejki, tworzyć statystykę słów i wypisywać wynik. Ostateczna implementacja wygląda tak:

```
221 while (!Thread.currentThread().isInterrupted()) {
222     try {
223         // TODO pobieranie ścieżki (Optional<Path> optPath = kolejka.take()); i tworzenie statystyki wyrazów
224         Optional<Path> optPath = kolejka.take();
225         if (!optPath.isPresent()) {
226             System.out.printf("KONSUMENT %s otrzymał poison pill - kończy pracę.%n", name);
227             break;
228         }
229         Path path = optPath.get();
230         System.out.printf("KONSUMENT %s przetwarza plik: %s%n", name, path);
231
232         Map<String, Long> countedWords =
233             statisticsService.getLinkedCountedWords(path, liczbaWyrazowStatystyki);
234
235         System.out.printf("Najczęstsze słowa w pliku %s (konsument %s):%n", path, name);
236         countedWords.forEach((word, count) ->
237             System.out.printf("%s = %d%n", word, count));
238
239     } catch (InterruptedException e) {
240         info = String.format("Oczekiwanie konsumenta %s na nowy element z kolejki przerwane!", name);
241         System.out.println(info);
242         Thread.currentThread().interrupt();
243     }
244 }
245 String infoEnd = String.format("KONSUMENT %s ZAKOŃCZYŁ PRACĘ", name);
246 System.out.println(infoEnd);
```

Konsument w pętli wywołuje `kolejka.take()`, co blokuje go, jeśli aktualnie nie ma żadnych elementów w kolejce. Jeśli pobierze `Optional.empty()`, traktuje to jako poison pill, wypisuje stosowny komunikat i wychodzi z pętli. W przeciwnym przypadku pobiera `Path`, wywołuje `statisticsService.getLinkedCountedWords()` i wypisuje w konsoli listę słów wraz z częstotliwością ich występowania.

2.5. Przetwarzanie tekstu- metoda getLinkedCountedWords i strumienie

Do tworzenia statystyk słów używam klasy WordStatisticsService i metody getLinkedCountedWords(Path path, int wordsLimit). Tutaj wykorzystałem API strumieni oraz wyrażenia lambda.

Implementacja metody getLinkedCountedWords wygląda tak:

```
20 public Map<String, Long> getLinkedCountedWords(Path path, int wordsLimit) {
21     try (BufferedReader reader = Files.newBufferedReader(path)) {
22         return reader.lines()
23             // 1. Podział linii na słowa
24             .map(line -> line.split("\\s+"))
25             .flatMap(Arrays::stream)
26             // 2. Konwersja do małych liter (żeby uprościć regex)
27             .map(String::toLowerCase)
28             // 3. Wycięcie wszystkich znaków, które nie tworzą słów
29             .map(word -> word.replaceAll("[^a-z0-9ąęóśźń]+", ""))
30             // 4. Filtrowanie słów - tylko z przynajmniej trzema znakami
31             .filter(word -> word.matches("[a-z0-9ąęóśźń]{3,}"))
32             // 5. Grupowanie słów względem liczności ich występowania
33             .collect(Collectors.groupingBy(
34                 Function.identity(),
35                 Collectors.counting()))
36             .entrySet()
37             .stream()
38             // 6. Sortowanie względem przechowywanych w mapie wartości, malejąco
39             .sorted(Map.Entry.comparingByValue(Comparator.reverseOrder()))
40             // 7. Ograniczenie liczby słów do wartości z wordsLimit
41             .limit(wordsLimit)
42             .collect(Collectors.toMap(
43                 Map.Entry::getKey,
44                 Map.Entry::getValue,
45                 (k, v) -> {
46                     throw new IllegalStateException(String.format("Błąd! Duplikat klucza %s.", k));
47                 },
48                 LinkedHashMap::new
49             ));
50     } catch (IOException e) {
51         throw new RuntimeException(e);
52     }
53 }
```

Poszczególne działania w tej metodzie:

1. Plik jest otwierany za pomocą Files.newBufferedReader(path), konstrukcja try-with-resources automatycznie zamyka strumień.
2. reader.lines() zwraca strumień linii tekstu.
3. Każda linia jest dzielona na tablicę słów przy pomocy split("\\s+"), która jest następnie spłaszczana do jednego strumienia słów przez flatMap(Arrays::stream).
4. Wszystkie słowa są zamieniane na małe litery (String::toLowerCase), żeby statystyka była niewrażliwa na wielkość liter.
5. Wszystkie znaki, które nie są literami lub cyframi, są usuwane, z uwzględnieniem polskich znaków.
6. Filtrowanie wyłapuje tylko słowa, które mają co najmniej 3 znaki (word.matches("[a-z0-9ąęóśźń]{3,}")).

7. Grupowanie `Collectors.groupingBy(Function.identity(), Collectors.counting())` zlicza wystąpienia każdego unikalnego słowa i zwraca mapę `Map<String, Long>`.
8. Następnie wpisy tej mapy (`entrySet().stream()`) są zamieniane na strumień, sortowane malejąco po wartości (`Comparator.reverseOrder()`) i ograniczane do `wordsLimit` najczęstszych słów.
9. Na końcu wynik jest zbierany do `LinkedHashMap`, żeby zachować posortowaną kolejność w końcowej mapie.

Taka implementacja ze strumieniami jest czytelniejsza od tradycyjnych pętli `for` i ręcznego wypełniania map, a także bardziej zwięzła.

2.6. Zadanie dodatkowe- wyznaczanie podobieństwa kosinusowego

Dodatkowa część zadania polegała na utworzeniu wersji programu tak, aby zamiast tylko liczyć słowa, porównywała każdy znaleziony dokument z jednym dokumentem wzorcowym przy użyciu miary kosinusowej. W tym celu rozbudowałem `MainFrame` o metodę `getCosineSimilarityStatistics()` i dodałem przycisk „Cosine” w GUI.

2.6.1. Obsługa producenta w zadaniu dodatkowym

Producent w tej wersji programu jest podobny do tego z zadania podstawowego, ale dodatkowo omija plik wzorcowy (`pattern.txt`), żeby nie dodawać go do kolejki:

```
165         final Path patternPath = Paths.get(DIR_PATH, "pattern.txt");

191     PathMatcher matcher = FileSystems.getDefault().getPathMatcher("glob:**/*.txt");
192     Files.walkFileTree(dir, new SimpleFileVisitor<Path>() {
193         @Override
194         public FileVisitResult visitFile(Path file, BasicFileAttributes attrs)
195             throws IOException {
196             if (matcher.matches(file)) {
197                 // pominięcie dokumentu wzorcowego
198                 if (file.equals(patternPath)) {
199                     return FileVisitResult.CONTINUE;
200                 }
201                 try {
202                     Optional<Path> optPath = Optional.ofNullable(file);
203                     kolejka.put(optPath);
204                     System.out.printf("PRODUCENT(cosine) %s dodał do kolejki: %s\n", name, file);
205                 } catch (InterruptedException e) {
206                     System.out.printf("PRODUCENT(cosine) %s przerwany podczas put()\n", name);
207                     Thread.currentThread().interrupt();
208                     return FileVisitResult.TERMINATE;
209                 }
210             }
211             return FileVisitResult.CONTINUE;
212         }
213     });
```


Obsługa fajrant i poison pills jest analogiczna do wersji podstawowej - tutaj również po zatrzymaniu producent wysyła `Optional.empty()` do kolejki tyle razy, ile jest konsumentów.

2.6.2. Konsument - liczenie podobieństwa kosinusowego

Konsument w „wersji kosinusowej” pobiera ścieżki z kolejki i dla każdego pliku liczy podobieństwo kosinusowe względem `patternPath`:

```
242     while (!Thread.currentThread().isInterrupted()) {
243         try {
244             Optional<Path> optPath = kolejka.take();
245             if (!optPath.isPresent()) {
246                 System.out.printf("KONSUMENT(cosine) %s otrzymał poison pill - kończy pracę.%n", name);
247                 break;
248             }
249             Path path = optPath.get();
250             System.out.printf("KONSUMENT(cosine) %s przetwarza plik: %s%n", name, path);
251
252             double similarity = statisticsService.cosineSimilarity(path, patternPath);
253
254             System.out.printf("Podobieństwo kosinusowe pliku %s do wzorca %s (konsument %s): %.4f%n",
255                             path, patternPath, name, similarity);
256
257         } catch (InterruptedException e) {
258             info = String.format("Oczekiwanie konsumenta(cosine) %s na nowy element z kolejki przerwane!",
259                                 name);
260             System.out.println(info);
261             Thread.currentThread().interrupt();
262         }
263     }
264     String infoEnd = String.format("KONSUMENT(cosine) %s ZAKOŃCZYŁ PRACĘ", name);
```

Klasa `WordStatisticsService` udostępnia metodę pomocniczą:

```
65     // pomocnicza metoda do policzenia podobieństwa jednego pliku do drugiego
66     public double cosineSimilarity(Path doc, Path pattern) {
67         Map<String, Long> docCounts = getFullWordCounts(doc);
68         Map<String, Long> patternCounts = getFullWordCounts(pattern);
69         return cosineSimilarity(docCounts, patternCounts);
70     }
```

Metoda `getFullWordCounts()` zwraca pełną mapę słów dla podanego dokumentu, wykorzystując te same kroki czyszczenia i filtrowania co `getLinkedCountedWords`, tylko bez sortowania i ograniczania limitu.

2.6.3. Obliczanie miary kosinusowej

Obliczanie miary kosinusowej jest zaimplementowane jako:

```
34 public double cosineSimilarity(Map<String, Long> a, Map<String, Long> b) {
35     // iloczyn skalarny
36     double dot = 0.0;
37     for (Map.Entry<String, Long> e : a.entrySet()) {
38         Long bv = b.get(e.getKey());
39         if (bv != null) {
40             dot += e.getValue() * bv;
41         }
42     }
43
44     // iloczyn norm
45     // norma wektora a
46     double normA = 0.0;
47     for (Long v : a.values()) {
48         normA += v * v;
49     }
50     normA = Math.sqrt(normA);
51
52     // norma wektora b
53     double normB = 0.0;
54     for (Long v : b.values()) {
55         normB += v * v;
56     }
57     normB = Math.sqrt(normB);
58
59     if (normA == 0.0 || normB == 0.0) {
60         return 0.0;
61     }
62     return dot / (normA * normB);
63 }
```

Mapy a i b są traktowane jako wektory, w których klucze to słowa, a wartości to liczba ich wystąpień.

Najpierw wyliczany jest iloczyn skalarny, przechodząc po wszystkich słowach z mapy a i dla każdego sprawdzając, czy występuje w mapie b; jeśli tak, to jest dodawany do sumy zliczoneA * zliczoneB.

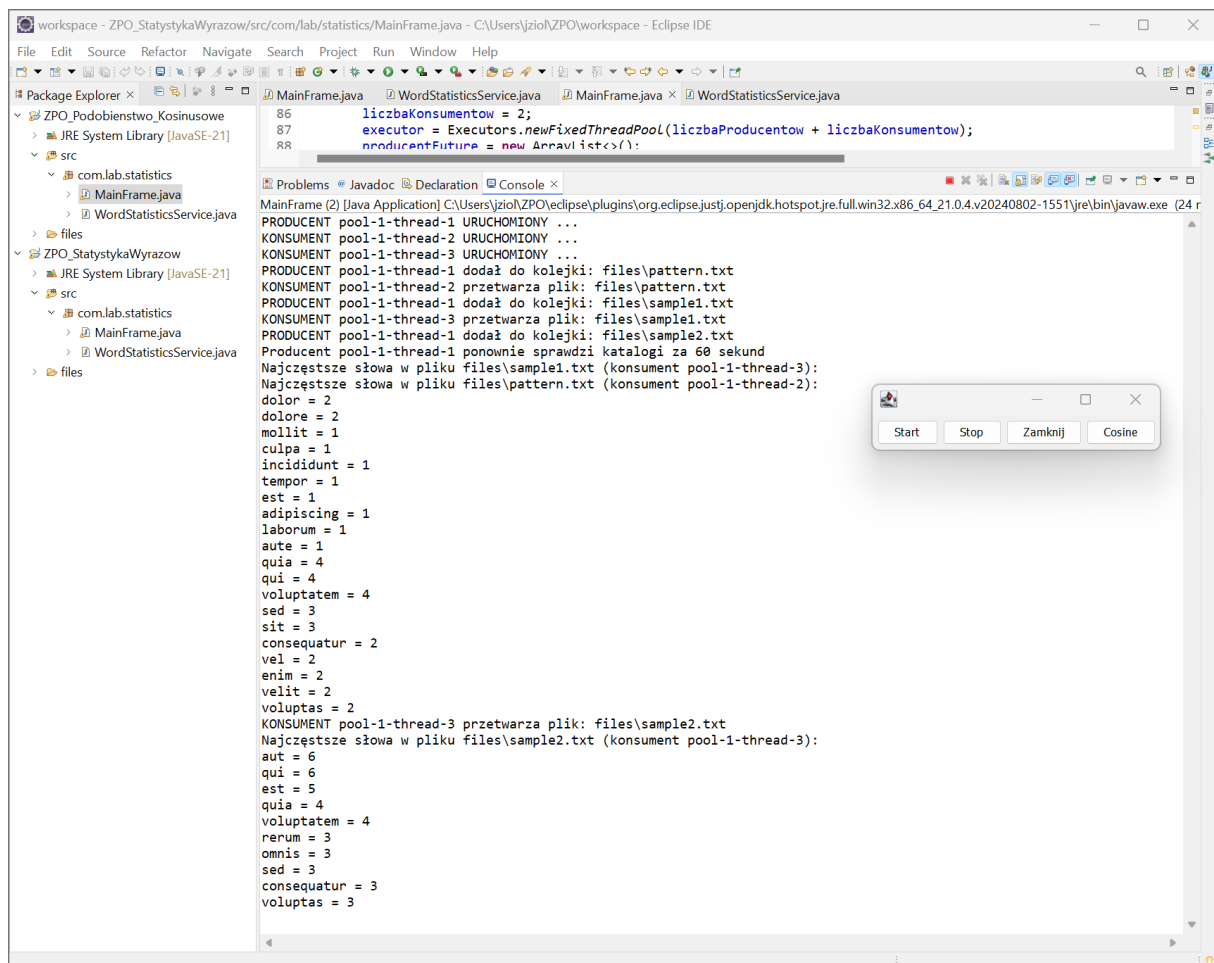
Następnie wyliczane są normy wektorów jako pierwiastek z sumy kwadratów częstotliwości. Na końcu metoda zwraca $\text{dot} / (\text{normA} * \text{normB})$ z walidacją przypadku, w którym jeden z wektorów ma normę zero (wtedy podobieństwo to 0). Dzięki temu wartość podobieństwa jest zawsze w zakresie od 0 do 1; im większa, tym bardziej podobne są dokumenty.

2.7. Interfejs graficzny

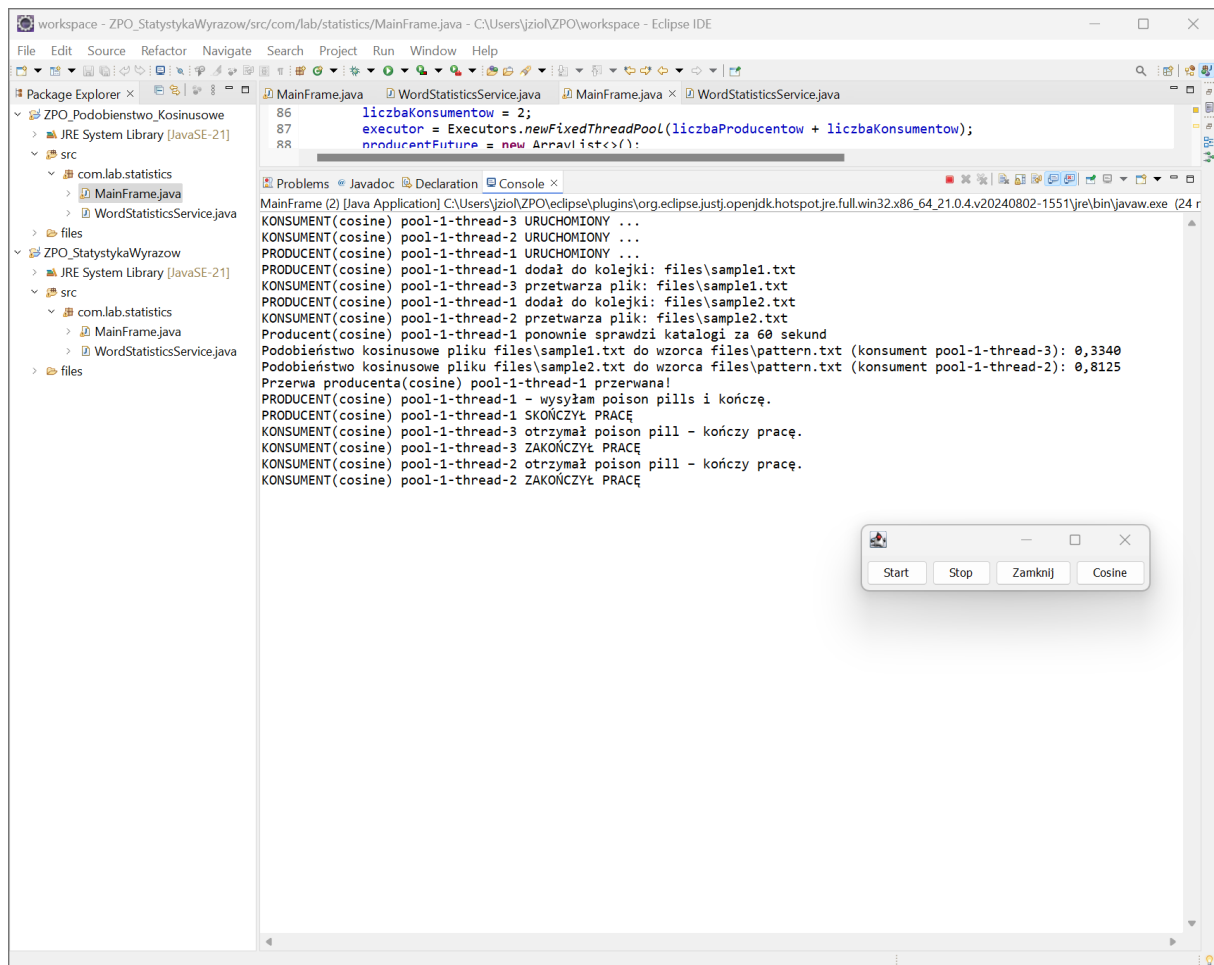
Interfejs graficzny jest oparty o Swing i znajduje się w klasie MainFrame. Okno zawiera panel z czterema przyciskami:

- 1) Start - uruchamia wariant podstawowy, czyli zliczanie słów w plikach.
- 2) Stop - ustawia fajrant na true i przerywa producentów, co w efekcie powoduje wysłanie poison pills do konsumentów.
- 3) Cosine - uruchamia program z podobieństwem kosinusowym.
- 4) Zamknij – przerywa wszystkie wątki (`executor.shutdownNow()`) i zamyka okno.

Wszystkie komunikaty diagnostyczne zostały skierowane do konsoli systemowej (`System.out.println()`), co ułatwia podejrzanie działania aplikacji podczas testów w IDE.



Rysunek 1 - wynik w konsoli po uruchomieniu programu i naciśnięciu Start



Rysunek 2 - wynik w konsoli po uruchomieniu programu i naciśnięciu Cosine, a następnie Stop

3. Wnioski końcowe (Podsumowanie)

W trakcie wykonywania tego laboratorium nauczyłem się wiele zarówno o programowaniu współbieżnym, jak i o strumieniach w Javie. Dowiedziałem się, jak wzorzec producent–konsument wraz z kolejką blokującą pozwala bezpiecznie wymieniać dane między wątkami, a mechanizm poison pills pozwala w prosty sposób zakończyć wątki konsumentów, nie używając takich funkcji, jak stop().

Natomiast implementacja WordStatisticsService dotyczyła bardziej programowania funkcyjnego i strumieni. Jedna metoda getLinkedCountedWords w sposób deklaratywny czyta plik, dzieli go na słowa, czyści je, filtruje, grupuje, sortuje i ogranicza wynik (do zadanego limitu). Taki kod ze strumieniami jest nie tylko krótszy, ale też łatwiej go modyfikować, ponieważ wystarczy dodać kolejną operację pośrednią.

Wersja programu z podobieństwem kosinusowym była przykładem tego, jak można reprezentować tekst jako wektor, i jak obliczać podobieństwo między dokumentami

matematycznie. Pomimo tego, że sama formuła jest dość prosta, to w praktyce trzeba dobrze przemyśleć wcześniejsze reguły czyszczenia, filtrowania itd., żeby wyniki miały sens.

Podsumowując, cel laboratorium został spełniony. Pomyślnie zrealizowałem następujące kroki:

- Implementacja wzorca producent–konsument z kolejką blokującą i poison pills.
- Rozdzielenie logiki GUI od logiki przetwarzania danych (klasy WordStatisticsService).
- Wykorzystanie strumieni i wyrażeń lambda, podnoszących czytelność i zwięzłość kodu.
- Rozszerzenie aplikacji o wariant z miarą kosinusową.